SENA · Análisis y Desarrollo de Software · Guía 8

Codivis Móvil

APK (desarrollar módulos móviles según requerimientos del proyecto).

Evidencia GA8-220501096-AA2-EV02

Tatiana Guerrero · 18 de octubre de 2026

1. Propósito y alcance

Desarrollar módulos Android del CRM Codivis para registrar cuentas, autenticar usuarios y administrar sus clientes desde un teléfono. Esta implementación funciona sin conexión mediante SQLite privado. Cada cuenta accede únicamente a sus clientes. No sincroniza con la API web de la Guía 7; es una implementación móvil local, con acceso a datos documentado y sustituible mediante la interfaz Repositorio.

El paquete com.codivis.crm.movil es independiente de la primera aplicación de EV01. Ambas se pueden instalar juntas. No hay cuentas ni clientes precargados.

2. Requerimientos y criterios de aceptación

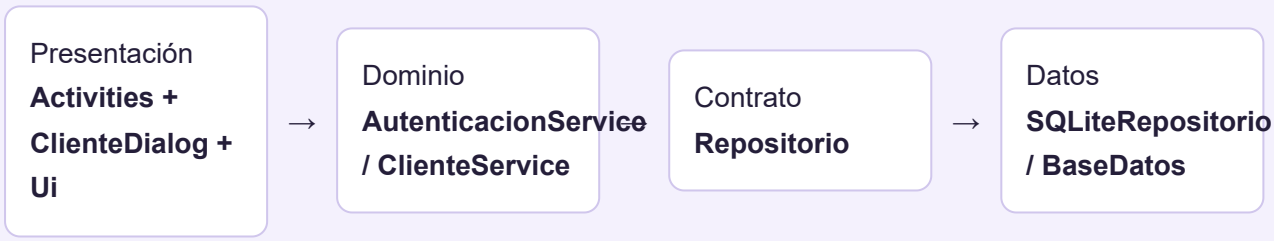
Código	Requerimiento	Comprobación
RF01	Registrar una cuenta con usuario y contraseña confirmada.	Usuario normalizado, único, de 3 a 40 caracteres; contraseña de 8 a 128.
RF02	Iniciar y cerrar sesión.	Credenciales correctas abren clientes; cierre elimina sesión y limpia navegación.
RF03	Proteger los módulos de clientes.	Sin sesión se redirige al acceso; operaciones sin identificador válido se rechazan.
RF04	Crear y listar clientes propios.	Nombre, correo y teléfono opcional; correo único por cuenta.
RF05	Buscar, editar y eliminar clientes.	Búsqueda por nombre/correo; edición conserva identificador; borrado requiere confirmación.
RF06	Conservar información al reabrir la aplicación.	SQLite conserva usuarios/clientes; preferencias privadas conservan sesión.

Código	Requerimiento	Comprobación
RNF01	Separar datos de cada cuenta.	Consultas, cambios y eliminaciones incluyen usuario_id.
RNF02	Evitar contraseñas en texto plano.	PBKDF2-HMAC-SHA256, sal aleatoria, 600000 iteraciones y comparación de hash.
RNF03	Interfaz reutilizable y operaciones fuera del hilo principal.	Ui, BaseActivity y ClienteDialog; ejecutor de un hilo para datos y credenciales.

3. Metodología y arquitectura

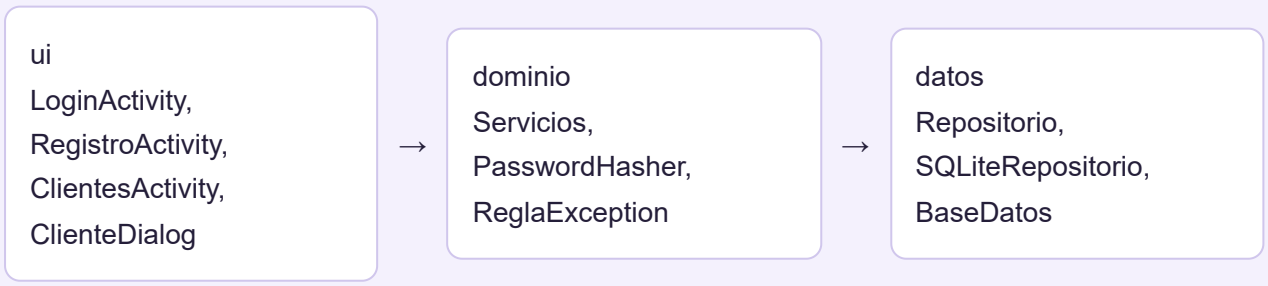
Se empleó un desarrollo incremental: definición de alcance y validaciones; implementación de dominio y persistencia; construcción de pantallas; pruebas automatizadas; compilación y documentación. No se atribuyen reuniones o ceremonias de equipo que no se hayan realizado. Los diagramas siguientes describen el código de esta entrega.

Diagrama de componentes y capas



CodivisApp compone las dependencias. Sesion conserva el identificador y nombre en preferencias privadas. Se aplica Repository para separar reglas y acceso a datos, composición de dependencias y reutilización de formularios. Las Activities coordinan eventos; las reglas no dependen de los widgets.

Diagrama de paquetes



Paquetes de apoyo

modelo
Usuario, Cliente → sesion
Sesion → raíz
CodivisApp

Diagrama de clases: relaciones principales

```
CodivisApp ◆── AutenticacionService ──→ Repositorio
           ◆── ClienteService ──────────→ Repositorio
           ◆── Sesion
SQLiteRepositorio ──▷ Repositorio (implementación)
SQLiteRepositorio ◆── BaseDatos (SQLiteOpenHelper)
AutenticacionService ──→ PasswordHasher
Usuario { id, nombre, claveHash }
Cliente { id, usuarioId, nombre, correo, telefono }
Usuario 1 ──────────── 0..* Cliente
BaseActivity ◁── LoginActivity / RegistroActivity / ClientesActivity
ClientesActivity ──→ ClienteDialog (crear y editar)
```

Mapa de navegación

Inicio → Acceso
Si existe sesión → Clientes
→ Acceso ↔ Registro
Registro correcto → Acceso
→ Clientes ↔ Crear / Editar
Eliminar → Confirmación
→ Cerrar sesión → Acceso (pila limpia)

4. Implementación de módulos

Módulo	Archivos y comportamiento
Autenticación	AutenticacionService valida datos y duplicados; PasswordHasher produce y verifica hashes. RegistroActivity y LoginActivity muestran errores sin descartar el formulario.
Clientes	ClienteService concentra validaciones y propiedad. ClientesActivity lista y busca; ClienteDialog reutiliza creación y edición; eliminación con confirmación.

Módulo	Archivos y comportamiento
Persistencia	BaseDatos crea tablas y activa claves foráneas. SQLiteRepositorio utiliza argumentos enlazados y filtros de propietario, y cierra cursores.
Sesión	Sesion guarda solo id y nombre. No almacena contraseña ni hash en preferencias. BaseActivity exige sesión para la pantalla de clientes.
Componentes visuales	Ui comparte campos, botones, mensajes y espaciados. Se utilizan widgets del framework Android, sin librerías externas de interfaz.

Datos y configuración de servidor

No requiere servidor, puerto, credenciales de base de datos externa ni permiso de Internet. SQLiteOpenHelper administra codivis.db, versión 1, en el almacenamiento privado de la aplicación. usuarios contiene id, nombre único sin distinguir mayúsculas y clave_hash. clientes contiene id, usuario_id, nombre, correo y teléfono, con clave foránea y restricción única (usuario_id, correo). Se indexa el propietario.

La sesión permanece al cerrar y volver a abrir hasta pulsar Cerrar sesión. Desinstalar o borrar los datos elimina las cuentas locales. El APK no ofrece recuperación de contraseña ni sincronización. Para versiones futuras se deben implementar migraciones explícitas; el código no elimina tablas silenciosamente ante cambios de versión.

5. Entornos de desarrollo y pruebas

Elemento	Configuración
Sistema anfitrión	Windows; JDK 25 disponible para ejecutar Gradle. Código fuente compilado con nivel Java 17.
IDE / herramientas	Android Studio; Gradle Wrapper 9.1.0; Android Gradle Plugin 9.0.1; SDK Android y Build Tools 36.0.0.
Compatibilidad declarada	minSdk 26 (Android 8.0); compileSdk y targetSdk 35. La declaración mínima no sustituye una matriz de pruebas en dispositivos.
Bibliotecas	Framework Android nativo y SQLite. Pruebas: JUnit 4.13.2 y Robolectric 4.17.
Prueba automatizada	Robolectric con API 28 para base de datos, sesión e interfaz; pruebas de hash en JVM.
Prueba física	Pendiente: no se ha instalado y ejecutado esta versión en un teléfono. El emulador del equipo no quedó operativo por limitaciones de virtualización.
Compilado	APK de depuración firmado automáticamente, para evaluación e instalación manual; no publicación en tienda.

6. Pruebas y resultados reales

37 pruebas ejecutadas; 0 fallos o errores. Se ejecutaron testDebugUnitTest y assembleDebug con resultado BUILD SUCCESSFUL. Los XML originales y el registro de compilación se incluyen en evidencias. Robolectric ejecuta pruebas Android en JVM; estos resultados no son capturas ni pruebas realizadas en un teléfono.

com.codivis.movil.DatosYDominioTest · 27 pruebas

- autenticaUnaCuentaPersistida
- listaSoloClientesDelUsuario
- eliminaYNoSePuedeVolverAConsultar
- loginRechazaClaveIncorrecta
- otraCuentaNoPuedeConsultarEditarOEliminar
- permiteTelefonoVacio
- sinSesionNoPuedeOperar
- baseImpideClienteConPropietarioInexistente
- noPermiteUsuarioDuplicado
- registroNormalizaYProtegeLaClave
- actualizaYConservaId
- validaCorreo
- permiteMismoCorreoEnCuentasSeparadas
- puedeEditarSinCambiarCorreo
- validaLongitudDeUsuario
- loginRechazaUsuarioInexistente
- noActualizaConCorreoDeOtroCliente
- validaTelefono
- validaCaracteresDeUsuario
- conservaClientesAlReabrirBase
- validaNombreObligatorio
- validaConfirmacion
- creaClienteConDatosNormalizados
- impideCorreoDuplicadoEnMismaCuenta
- loginRechazaDatosVacios
- validaClaveCorta
- buscaPorNombreYCorreo

com.codivis.movil.PasswordHasherTest · 4 pruebas

- salAleatoriaProduceHashesDiferentes
- aceptaClaveCorrectaYRechazaIncorrecta
- noAceptaClaveNulaONombresDeAlgoritmoManipulados
- rechazaHashDeterioradoSinExcepcion

com.codivis.movil.SesionYUiTest · 6 pruebas

- pantallaPrivadaSinSesionRedirigeAlLogin
- loginCorrectoAbreClientes
- registroInvalidoMuestraErrorYSigueEditable
- cerrarSesionEliminaIdentidad
- sesionRecuerdaIdentidadSinGuardarClave
- listadoMuestraClientesYBotonSalirCierraSesion

Comprobación manual pendiente en Android

1. Instalar el APK y abrir Codivis Móvil. Crear el usuario pedro con una contraseña propia de al menos 8 caracteres.
2. Iniciar sesión; crear un cliente con nombre, correo y teléfono opcional.
3. Buscarlo, editarlo y comprobar persistencia al reabrir.
4. Cerrar sesión, registrar luis y comprobar que no ve los clientes de pedro.
5. Volver a pedro, eliminar el cliente con confirmación y cerrar sesión.

7. Apertura, compilación e instalación

1. Abrir esta carpeta completa en Android Studio (contiene settings.gradle). Instalar SDK 35 y Build Tools 36.0.0. Seleccionar un JDK compatible con Gradle 9.1.
2. Permitir que Android Studio configure local.properties con la ruta del SDK del equipo. Ese archivo no se distribuye.
3. Ejecutar `gradlew.bat testDebugUnitTest assembleDebug`. La primera ejecución requiere Internet para descargar herramientas y dependencias.
4. El resultado se genera en `app/build/outputs/apk/debug/app-debug.apk`. Una copia se entrega en `apk/Codivis-Movil-GA8-AA2-EV02.apk`.
5. Copiar el APK al teléfono, abrirlo y permitir la instalación desde esa fuente si Android lo solicita. Alternativamente, con depuración USB autorizada: `adb install -r apk/Codivis-Movil-GA8-AA2-EV02.apk`.

8. Control de versiones y entrega

Repositorio Git del proyecto: https://github.com/taticodivis/Tatiana_guerrero_GA8_AA2_EV02. Se versionan código, APK, documentación y resultados. La carpeta evidencias contiene la comprobación de publicación de la versión entregada.

El archivo RAR reúne código fuente y Gradle Wrapper, APK, este documento en HTML/PDF e informes de prueba. Excluye SDK, cachés, rutas locales, claves de firma y carpetas generadas. La guía admite RAR o ZIP y pide código fuente, compilados, documentos y URL; no exige video en EV02.

9. Referencias

- Guía de aprendizaje 8, SENA, apartado GA8-220501096-AA2-EV02 (archivo suministrado en Documentos).
- Android Developers: arquitectura de aplicaciones.
- Android Developers: capa de datos.
- Android Developers: almacenamiento SQLite.
- Robolectric: configuración y pruebas.

Los resultados técnicos corresponden al código y a los informes incluidos en esta entrega.